

Pandas

Crash course

António Anjos
aanjos@uevora.pt

Departamento de Informática
Universidade de Évora

INF13262M – Applied Artificial Intelligence



1 Pandas



Motivation

- To avoid the need to write boilerplate code to load data from files
- To provide a set of tools to manipulate and analyse data

Warning!

Pandas is a very large library. This is just a starting point.



Pandas

- Pandas is a Python library for data manipulation and analysis
- It provides data structures and operations for manipulating numerical tables and time series
- It is built on top of NumPy
- It is one of the most popular Python libraries for data science

Importing pandas

```
import pandas as pd
```



Load data from a file

- `df = pandas.read_csv("filename.csv")`
- `df = pandas.read_csv("filename.zip")` *# if only 1 file inside*
- `df = pandas.read_excel("filename.xlsx")`
- `df = pandas.read_json("filename.json")`
- ...

Returns a DataFrame

- A dataframe is a table-like data structure
- Each column is a Series
- Each row is an observation



Example of a dataframe

	SepalLength	SepalWidth	PetalLength	PetalWidth	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
⋮	⋮	⋮	⋮	⋮	⋮
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
⋮	⋮	⋮	⋮	⋮	⋮



Getting information about the Dataframe

- `df.head()` – first 5 rows (default)
- `df.tail()` – last 5 rows (default)
- `df.shape` – number of rows and columns (doesn't include the header)
- `df.columns` – column names
- `df.info()` – information on the dataframe (shows a kind of schema)
- `df.describe()` – descriptive statistics (mean, std, quartiles, ...)
- `df.corr()` – correlation between columns



Selecting columns

- `col = df["column_name"]` *# DataFrame*
- `col = df.column_name` *# Series (same as col.column_name)*
- `cols = df[["column_name1", "column_name2"]]` *# DataFrame*
- `col1 = cols.column_name1`



Selecting rows with slicing

- `df[0:5]` – first 5 rows
- `df[-5:]` – last 5 rows
- `df[0:5:2]` – first 5 rows, every 2 rows
- `df[::-1]` – all rows, in reverse order

Colon is required!

- `df[0]` – error



Selecting subsets of rows and columns

- `df.iloc[0]` – first row (colon is not required)
- `df.iloc[0:5]` – first 5 rows
- `df.iloc[-1]` – last row
- `df.iloc[-5:]` – last 5 rows
- `df.iloc[0:5, 0:2]` – first 5 rows and first 2 columns
- `df.iloc[0:5, [0, 2]]` – first 5 rows and columns 0 and 2
- `df.iloc[0:5, -2:]` – first 5 rows and last 2 columns

Only numeric indices!

- `df.iloc["column_name"]` – error!
- Explore `df.loc` to be able to use column names

Checking for invalid values

- `df.isnull()` – returns a dataframe with True/False values
- `df.isnull().sum()` – returns the number of invalid values per column (or row if `axis=1`)
- `df.isnull().any()` – returns True if there are invalid values in the column (i.e., for a column, at least one row has an invalid value)
- `df.isnull().any(axis=1)` – returns True if there are invalid values in the row (i.e., for a row, at least one **column** has an invalid value)

axis

- Determines the direction in which the operation is performed
- `axis=0` (default) – in the row direction
- `axis=1` – in the column direction

Dealing with missing values

- `df = df.dropna()` – drop rows with missing values
- `df = df.dropna(axis=1)` – drop columns with missing values
- `df = df.fillna(0)` – replace missing values with 0
- `df = df.fillna(df.mean())` – replace missing values with the mean of the column
- `df = df.drop("column_name", axis=1)` – drop column
- `df = df.drop(6)` – drop row with index 6
- `df.col_name = df.col_name.replace("old", "new")` – replace values in a column



Getting numpy arrays

- `df.values` – returns a numpy array with all values in the dataframe
 - ▶ If there are columns with different types, the type of the array will be `object`
- `df["column_name"].values` – numpy array of specific column (n -D)
- `df.column_name.values` – numpy array of specific column (0-D)
- `df.iloc[0].values` – returns a numpy array with the values of the first row
- Notice that the header is not included in the numpy array

Why?

- numpy arrays are more efficient than dataframes
- numpy arrays are required by many machine learning frameworks (e.g., `scikit-learn`)



Aggregation of data

- `df.groupby("column_name")` – groups rows by similar values in given column
- `df.groupby("column_name").mean()` – aggregates by mean
- `df.groupby("column_name").sum()` – aggregates by sum
- `df.groupby("column_name").count()` – number of rows in each group
- `df.groupby("column_name").size()` – same as `count()`
- `df.groupby("column_name").agg(["mean", "sum"])` – returns the mean and sum of each column for each group



- `pd.to_datetime(df["column_name"])` – converts a column with date strings to a date format (interpretable as such by pandas)
- `pd.to_datetime(df["column_name"], format="%Y-%m-%d")` – converts column to dates with a specific format
- `pd.to_datetime(df["column_name"], dayfirst=False)` – convert column where dates start with the day
- `df["column_name"].to_numeric()` – converts column to numeric values
 - ▶ Useful when we need the date to work as a feature for regression



Other useful methods

- `df = df.sample(frac=1)`: shuffles the rows of the dataframe
- `df = df.sample(frac=0.2)`: randomly selects 20% of the rows of the df
- `df = df.sample(n=10)`: randomly selects 10 rows of the df
- `df['column_name'].nunique()`: number of unique values in a column
- `df['column_name'].unique()`: returns the unique values in a column
- `df = df.rename(columns={"old_name": "new_name"})`: rename a column



- https://pandas.pydata.org/docs/user_guide/index.html

